

# 核心算法描述

para\_alg\_impl.py 为对应 Python 实现

## 引言 / Introduction

本节概述 shuttle 参数搜索算法的目标、输入与输出。

- 输入参数为  $q, n, \ell, m, \sigma_1, \sigma_2, \alpha_h, \alpha_b$ 。
- 算法首先检查输入是否合法。
- 若输入合法，则计算 PkBytes、SignBytes 和 CombinedBytes。
- 随后基于 LWE 与 SIS 模型给出相应的安全估计。

为保证参数有意义，需要满足以下基本约束：

- $n$  取安全参数的 2 倍。
- $q$  为 NTT 素域模数。
- $\sigma_1 \geq 0.81, \sigma_2 \geq 0.81$ 。
- $\alpha_h$  为 2 的幂。

下文依次给出算法步骤、参数对象构造、尺寸公式，以及最终输出格式。

## 输入

输入为  $n, q, \ell, m, \sigma_1, \sigma_2, \alpha_h, \alpha_b$ 。其中手稿中的字母  $l$  在本文统一记为  $\ell$ ，以避免与对数符号  $\ln$  混淆。

## 算法步骤

1. 若  $n \notin \{256, 512, 1024\}$ ，则返回： $n$  不合法。
2.
$$\lambda = \frac{n}{2}.$$
3. 若  $\lambda \nmid (q-1)$ ，则返回： $q$  不合法， $n/2$  不整除  $(q-1)$ ， $q$  不是 NTT 素域。
4. 若  $\sigma_1 < 0.81$  或  $\sigma_2 < 0.81$ ，则返回： $\sigma_1, \sigma_2$  不合法，离散高斯中分布标准差太小。
5. 若  $\alpha_b$  不是 2 的非负整数次幂，则返回： $\alpha_b$  不合法，必须是二的幂次。

$$\sigma_b = \begin{cases} 0, & \alpha_b = 1, \\ \frac{4^{\log_2 \alpha_b + 2}}{12}, & \alpha_b \geq 2. \end{cases}$$

6.

$$\alpha_1 = \left\lceil 10\sqrt{n\sigma_1^2(\sigma_2^2 + \sigma_b)} \right\rceil$$

$$\alpha_s = \left\lceil 10\sqrt{\sigma_2^2 + \sigma_b} \right\rceil$$

$$\alpha_e = \lceil 10\sigma_1 \rceil$$

7.

$$B_k = \sqrt{\alpha_1^2 + \alpha_s^2 \ell n \sigma_1^2 + \alpha_e^2 m n \sigma_2^2}.$$

8. 根据  $\lambda$  取不同常数，计算

$$r = \begin{cases} \lceil 2.78B_k \rceil, & \lambda = 128, \\ \lceil 2.78B_k \rceil, & \lambda = 256, \\ \lceil 2.81B_k \rceil, & \lambda = 512. \end{cases}$$

9.

$$\mu_s = n\left(\frac{r}{\alpha_1}\right)^2 + \ell n\left(\frac{r}{\alpha_s}\right)^2 + mn\left(\frac{r}{\alpha_e}\right)^2 + \frac{m\alpha_h^2}{48}$$

10.

$$V_s = 2n\left(\frac{r}{\alpha_1}\right)^4 + 2\ell n\left(\frac{r}{\alpha_s}\right)^4 + mn\left(2\left(\frac{r}{\alpha_e}\right)^4 + \frac{\alpha_h^2\left(\frac{r}{\alpha_e}\right)^2}{12} + \frac{m\alpha_h^4}{2880}\right)$$

11.

$$B_v = B_s = \sqrt{\mu_s + 6.13\sqrt{V_s}}.$$

12. 若  $2r/\alpha_h < 0.05$  或者  $\alpha_h$  不整除  $2(q-1)$ ，则返回： $\alpha_h$  太大，请调整。

13. 若  $\alpha_h$  不是 2 的幂次，则返回： $\alpha_h$  不是 2 的幂次。

14. 若  $q$  小于  $2(q-1)$ ，则返回： $\alpha_h$  不整除  $2(q-1)$ 。

## 参数对象构造

`LWE_para = LWEParameters( $\ell n$ ,  $q$ ,  $X_s = D_{\sigma_1}$ ,  $X_e = D_{\sigma_2}$ ,  
 $mn$ , tag = None).`

`SIS_UF_para = SISParameters( $mn$ ,  $q$ , length_bound =  $B_v$ ,  
 $(\ell + 2 + m)n$ , norm = 2, tag = None).`

`SIS_sUF_para = SISParameters( $mn$ ,  $q$ , length_bound =  $2B_v$ ,  
 $(\ell + 1 + m)n$ , norm = 2, tag = None).`

## 尺寸公式

$$\text{PkBytes} = \frac{\lambda}{8} + \left\lceil \frac{nm \lceil \log_2 \frac{q}{\alpha_b} \rceil}{8} \right\rceil.$$

按图片中的定义，先记

$$\sigma_h := \frac{2r}{\alpha_h \cdot \alpha_e}.$$

$$A_h := 1 + 2e^{-\frac{1}{2\sigma_h^2}} + 2e^{-\frac{2}{\sigma_h^2}}.$$

$$H_h = \begin{cases} \frac{1}{2} \log_2(2\pi e \sigma_h^2), & \sigma_h \geq 1, \\ \log_2 A_h + \frac{e^{-\frac{1}{2\sigma_h^2}} + 4e^{-\frac{2}{\sigma_h^2}}}{\sigma_h^2 A_h \ln 2}, & \sigma_h < 1, \end{cases}$$

因为当前算法里种子贡献写作  $\lambda/8$  字节，所以与图片中的 SeedBytes 记号等价地写成，并对最终字节数向上取整

$$\text{SignBytes} = \left\lceil \frac{\lambda + \frac{n}{2} \log_2 \left( 2\pi e \left( \frac{r}{\alpha_1} \right)^2 \right) + \frac{\ell n}{2} \log_2 \left( 2\pi e \left( \frac{r}{\alpha_s} \right)^2 \right) + nm H_h}{8} \right\rceil.$$

## 组合尺寸与安全估计

继续按图片中的描述，组合尺寸定义为

$$\text{CombinedBytes} = \text{PkBytes} + \text{SignBytes}.$$

安全估计部分按如下方式计算：

- `LWE_security_bit` 由 `LWE.estimate.rough(LWE_para, quiet=True)` 给出，取其中两种攻击里最强攻击
- `SIS_UF_security_bit` 由 `SIS.estimate.rough(SIS_UF_para, quiet=True)` 给出，取对应的安全比特。
- `SIS_sUF_security_bit` 由 `SIS.estimate.rough(SIS_sUF_para, quiet=True)` 给出，取对应的安全比特。

## 最终输出

若输入参数合法，则最终输出可概括为如下 6 个量：

`(LWE_security_bit, SIS_UF_security_bit, SIS_sUF_security_bit, PkBytes, SignBytes, CombinedBytes)`。

也可以按图片里的表述写成一句结果说明：输入参数合法，得到如下结果。

```
LWE_security_bit = x,  
SIS_UF_security_bit = y,  
SIS_sUF_security_bit = z,  
PkBytes = a,  
SignBytes = b,  
CombinedBytes = w.
```

## 实现说明

本文对应的 Python 实现位于 `para_alg_impl.py`。脚本会完成上述合法性检查、推导全部中间量，并在本地存 `estimator` 依赖时构造 `LWEParameters` 与 `SISParameters` 对象。

直接运行示例：

```
python3 para_alg_impl.py --n 256 --q 13313 --l 3 --m 2  
--sigma-1 0.7071067811865476 --sigma-2 0.7071067811865476 --alpha-h 128
```